

ECE 315

Steven Knudsen, PhD, PEng

Lecture 6 :

- Embedded Software
- Design Thoughts
- Real-Time Specifications and Constraints



UNIVERSITY
OF ALBERTA

Last Lecture

- More on RTOS concepts via FreeRTOS
- Patterns
- FreeRTOS example showing two patterns

This Lecture

- Embedded Software
- Design Thoughts
- Real-Time Specifications and Constraints
 - Examples, Lessons

Learning Objectives

- Describe the iterative software development process
- Define a “walking skeleton”
- Describe the steps past the walking skeleton stage
- Understand events in an embedded context, and their sources
- Define real-time specifications like response time, variability, latency,...
- Describe why having processor idle time is a good idea
- Explain why sometimes software implementation of a signal may be a problem

Thoughts on Software Design and Development



UNIVERSITY
OF ALBERTA

5

Quotes

Frederick Brooks, author of *The Mythical Man-Month*

The hard part of building software is the specification, design, and testing of this conceptual construct, not the labor of representing it and testing the fidelity of the representation.

Douglas Crockford, developer of Javascript and more

Computer programs are the most complex things that humans make.

Grady Booch, author, pioneer of object-oriented design & programming

The function of good software is to make the complex appear to be simple

About Software Development

- Software development is often both expensive and risky
- Software is often impossible to specify completely at project start
- Software requirements are typically incomplete and often change
- Software systems can be arbitrarily complex
 - This complexity easily overwhelm the understanding and capabilities of teams of even expert software designers
- Turnover rates are high in the software industry.
 - Expertise is lost as designers leave. New designers must be trained, and
 - they are (at first) more likely to introduce software errors.



Adapted from ECE 315 W2025 by Dr. Bruce Cockburn

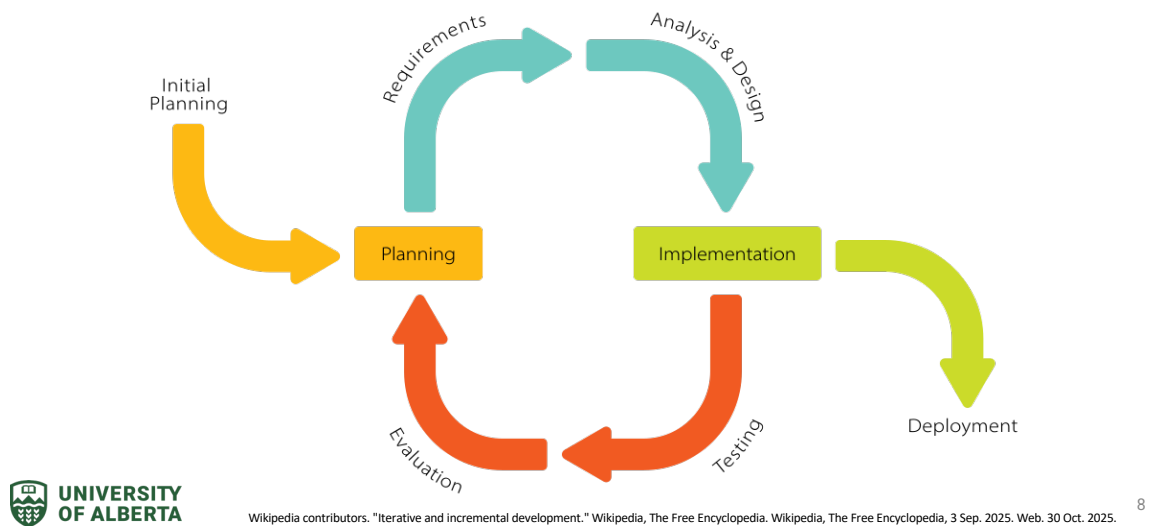
7

You are Computer Engineers and know that Software Engineering is a thing.

After 40+ years, and learning from people who were early pioneers and very smart, I can say that software development is far from an understood thing

Good software is hard, yet it looks so easy when done right

Iterative and Incremental Development



Can't expect to get something that is very complex right the first try.

Iterative, incremental programming is in almost all cases a good idea.

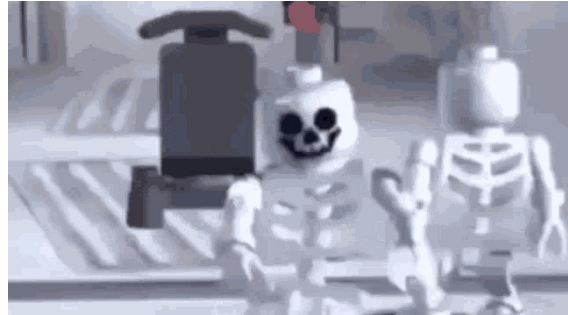
Engineering Design - Starting Simply

- Start with conceptual design, exploring the design space

Starting Simply – Walking Skeletons

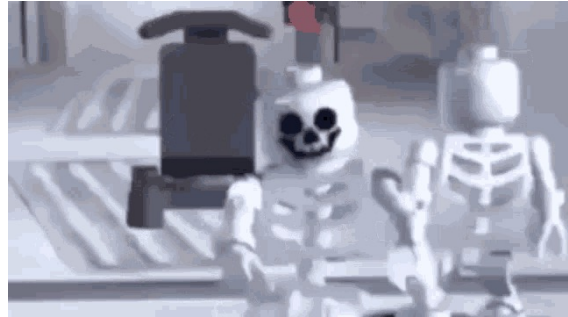
- Start with conceptual design, exploring the design space
- Choose a promising design that should meet the requirements
- Prove that it is viable
 - Define and test the minimum functionality needed to build the system

– “Walking Skeleton”



Starting Simply

- Prove that it is viable
 - Define and test the minimum functionality needed to build the system
 - “Walking Skeleton”
- High-level Design define & refine
- Consider Costs, Risks, and Mitigation strategies



Embedded Software ...

We will go into the philosophy and practice of embedded development in this course

It is a skill and getting experience is a good idea,

- ...as a developer to appreciate the challenges

- ...as a manager to appreciate the effort and time needed

- ...as an architect to never underestimate the complexity

Real-time Events

- Can be either external or internal to embedded system
- External event examples:
 - A signal has been received from outside the system, such as a user input or a communication message (e.g., Internet of Things message)
 - A sensor has detected a change in the environment.
 - An actuator has completed a commanded action
 - An alarm condition (e.g., power failure) has occurred
- Internal event examples:
 - A timer-triggered interrupt has occurred
 - A direct memory access data transfer has finished
 - A software-triggered exception has occurred

Real-time Specifications or Requirements

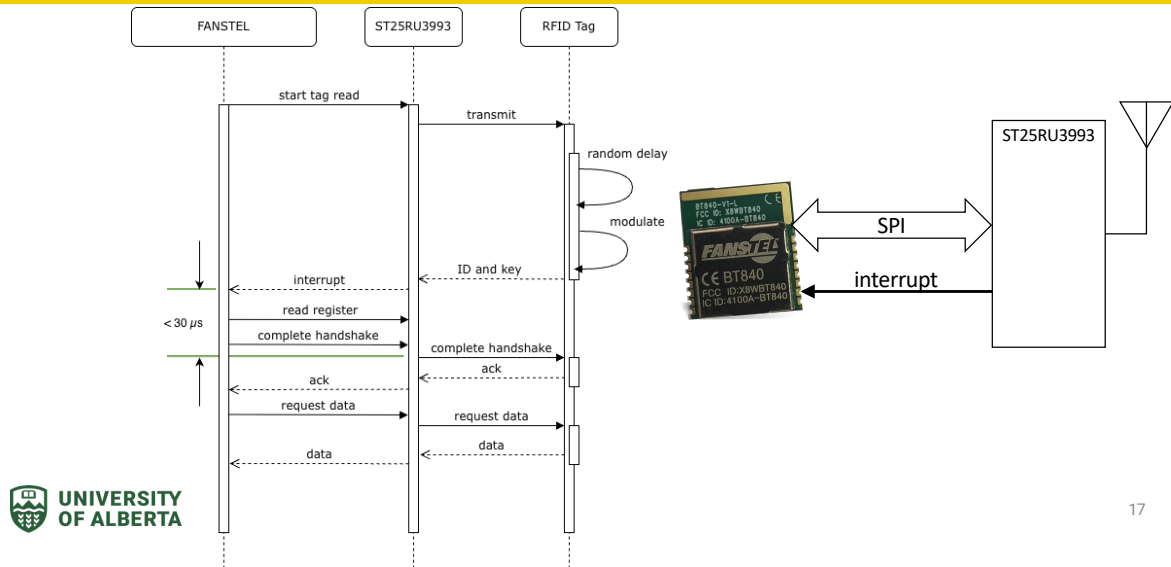
- Typical kinds of real-time specifications:
 - Maximum and/or minimum response time to different events
 - Maximum variability/jitter in the response time
 - Accuracy of the repetition frequency: e.g., sampling an analog sensor signal
 - Maximum variability/jitter in repetition frequency
 - Degradation behaviour when the workload exceeds system capacity
 - e.g., can less important activities be safely deferred (possibly by skipping over code or disabling interrupts) so that important events will be handled?

RFID Example

- RFID tags absorb energy from a transmission by a reader device
- The tag uses the energy to modulate the signal that is reflected by the antenna
- The reader demodulates the received reflection and gets the data contained
- How does the reader know which tag it is reading from and how does it differentiate it from all the others that might be around?



RFID Example



UML is a good thing, good for communicating ideas, processes, algorithms.

Here we use an interaction diagram to understand the handshaking involved in RFID tag reading

RFID Example



The diagram illustrates the internal architecture of the nRF52840 SoC. It features a central CPU core connected to multiple RAM banks (RAM0 through RAM9) and ROM blocks (ROM0 through ROM9). The CPU also interfaces with a GPU and a PS0-PS1/PS2-PS3 block. Peripheral controllers include TPU, SPI-DP, CTRN-AP, and ANB multilayer. The system is powered by VDD and VDDA pins.

- [illegible]

Idle time

- CPU execution time when no work or urgent work needs to be performed
- It provides timing flexibility, e.g.,
 - Interrupt service routines can more easily run without disrupting other tasks
 - Lowest priority jobs get a chance
 - Latency of event response is improved
- It can be used to perform lower-priority, non-real-time work such as:
 - File system maintenance
 - Dynamic memory defragmentation; data blocks in memory are moved to consolidate regions of unused memory.
- When idle time is used for low-priority work it is sometimes called background time.

Idle Time cont'd

Sufficient idle time must be present in real-time systems:

- For *externally initiated events*, idle time provides the necessary excess CPU capacity so that even worst-case bursts of event-handling workload can be handled within the maximum response time and determinism (i.e., maximum response time variability) specifications.
 - E.g., most of the time an interrupt service routine needs $N \mu s$, but sometimes it needs $\gg N \mu s$ – we must plan for and program for worst case situations

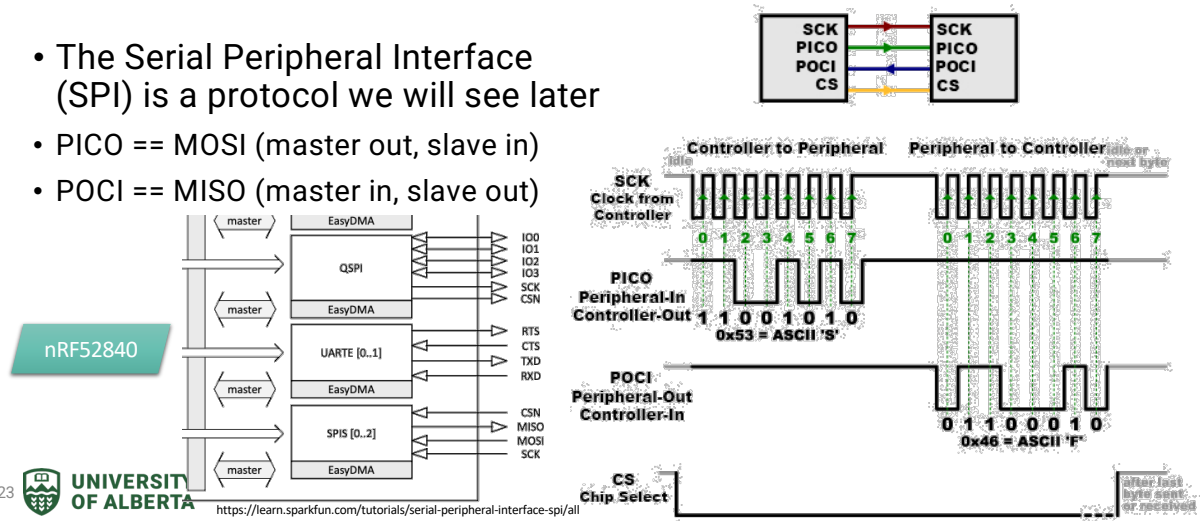
Idle Time cont'd

Sufficient idle time must be present in real-time systems:

- For *internally initiated events*, idle time provides flexibility so that the effects of internal sources of timing variability (e.g., variability in the execution time of compiled software, variability caused by cache memory and virtual memory misses, variability caused by Direct Memory Access transfers) can be absorbed and effectively hidden
- Note that the *timing for internally initiated events should be determined by H/W timers*, not by software-dependent execution delays.

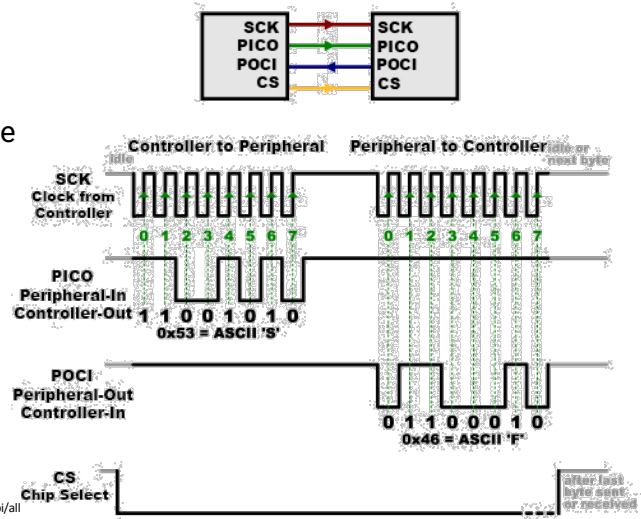
A Hard Lesson using Software Timing

- The Serial Peripheral Interface (SPI) is a protocol we will see later
- PICO == MOSI (master out, slave in)
- POCI == MISO (master in, slave out)



A Hard Lesson using Software Timing

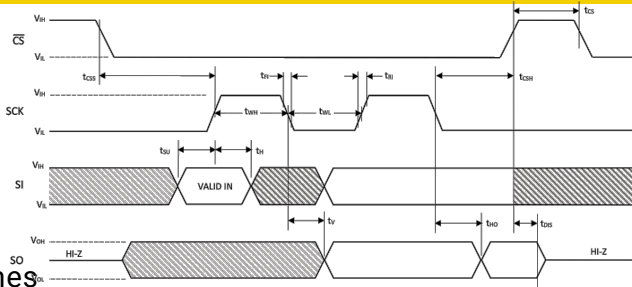
- Can implement using software and general purpose I/O (GPIO) pins
- Toggle GPIOs high and low to emulate the timing diagram
- The clock must be regular and at the right frequency
 - Processor clock must be fast enough to support this
- SPI clock speeds can be as high as 60 Mbps, but for embedded usually up to 12 Mbps



24

A Hard Lesson using Software Timing

- Timing constraints like t_{WH} and t_{WL} for the SCK signal must be adhered to
- Data on SI and SO is "clocked" on the rising edge of SCK
 - There is a set up time, t_{SU} , and a hold time, t_H that are minimum times
- A recent student-built space physics instrument failed when a processor connected to an analogue-to-digital converter (ADC) could not keep up using a software SPI implementation



Had to replace the processor(!)



UNIVERSITY
OF ALBERTA

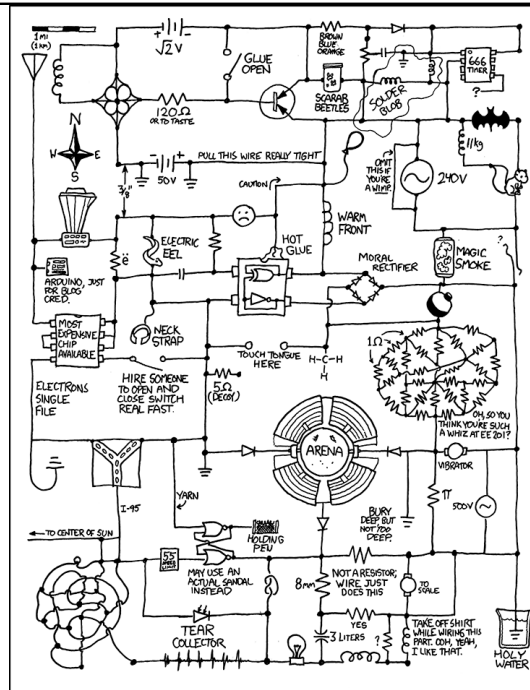
Microchip - TA101 CryptoAutomotive™ Summary Data Sheet

25

Next Lecture

- Bare Metal Programming
- More on RTOS scheduling

Questions



<https://kikid.com/730/>